

MapReduce Employee Contributions

Project Report

April 2026

1. Project Overview

Field	Value
Project Name	Employee Contributions MapReduce Application
Technology Stack	Java, Apache Hadoop MapReduce, Maven
Purpose	Aggregate employee contribution data using distributed computing

2. Project Setup (Maven Configuration)

2.1 Complete pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.hadoop.mapreduce</groupId>
  <artifactId>EmployeeContributions</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <dependencies>
    <!-- Hadoop Common Library -->
    <dependency>
      <groupId>org.apache.hadoop</groupId>
      <artifactId>hadoop-common</artifactId>
      <version>3.3.0</version>
    </dependency>
```

```

<!-- Hadoop MapReduce Client -->
<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-mapreduce-client-core</artifactId>
  <version>3.3.0</version>
</dependency>

<!-- Hadoop HDFS -->
<dependency>
  <groupId>org.apache.hadoop</groupId>
  <artifactId>hadoop-hdfs</artifactId>
  <version>3.3.0</version>
</dependency>

<!-- Logging Dependencies -->
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-api</artifactId>
  <version>1.7.30</version>
</dependency>

<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-log4j12</artifactId>
  <version>1.7.30</version>
</dependency>

<!-- JUnit for Testing -->
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.12</version>
  <scope>test</scope>
</dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.8.1</version>
      <configuration>
        <source>21</source>
        <target>21</target>
      </configuration>
    </plugin>
  </plugins>
</build>

</project>

```

2.2 Project Metadata

Field	Value
-------	-------

GroupId	com.hadoop.mapreduce
ArtifactId	EmployeeContributions
Version	1.0-SNAPSHOT
Packaging	jar

2.3 Dependencies

Dependency	Version	Scope
org.apache.hadoop : hadoop-common	3.3.0	compile
org.apache.hadoop : hadoop-mapreduce-client-core	3.3.0	compile
org.apache.hadoop : hadoop-hdfs	3.3.0	compile
org.slf4j : slf4j-api	1.7.30	compile
org.slf4j : slf4j-log4j12	1.7.30	compile
junit : junit	4.12	test

2.4 Build Configuration

The Maven Compiler Plugin (v3.8.1) is configured to target **Java 21** for both source and target compatibility.

3. Source Code Implementation

3.1 ContributionsMapper.java

The Mapper class extends `Mapper<Object, Text, Text, IntWritable>`. It reads each CSV line, skips the header, extracts the employee ID and contribution amount, and emits (`employeeId`, `contributionAmount`) key-value pairs.

```
package com.hadoop.mapreduce;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

import java.io.IOException;

public class ContributionsMapper extends Mapper<Object, Text, Text, IntWritable> {

    private final Text employeeId = new Text();
    private final IntWritable contributionAmount = new IntWritable();

    @Override
    protected void map(Object key, Text value, Context context)
        throws IOException, InterruptedException {

        String line = value.toString();
        String[] fields = line.split(",");

        // Skip header line
        if (!fields[0].equals("year")) {
            try {
                String id = fields[2].trim(); // Employee ID
                int amount = Integer.parseInt(fields[3].trim()); // Contribution Amount

                employeeId.set(id);
                contributionAmount.set(amount);

                context.write(employeeId, contributionAmount);
            } catch (NumberFormatException e) {
                System.err.println("Skipping invalid line: " + line);
            }
        }
    }
}
```

Item	Detail
Class	ContributionsMapper
Extends	Mapper<Object, Text, Text, IntWritable>
Input key type	Object (byte offset within input file)
Input value type	Text (one CSV line)

Output key type	Text (employee ID)
Output value type	IntWritable (contribution amount)

3.2 ContributionsReducer.java

The Reducer class extends `Reducer<Text, IntWritable, Text, IntWritable>`. It receives all contribution amounts grouped by employee ID, accumulates a running sum, and writes the total to the output context.

```
package com.hadoop.mapreduce;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

import java.io.IOException;

public class ContributionsReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    private final IntWritable outValue = new IntWritable();

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;
        for (IntWritable v : values) {
            sum += v.get();
        }

        outValue.set(sum);
        context.write(key, outValue);
    }
}
```

Item	Detail
Class	ContributionsReducer
Extends	Reducer<Text, IntWritable, Text, IntWritable>
Input key type	Text (employee ID)
Input value type	IntWritable (individual contribution amounts)
Output key type	Text (employee ID)
Output value type	IntWritable (total contributions)

3.3 ContributionsJob.java

The driver class wires the Mapper and Reducer together. It accepts two command-line arguments (HDFS input path and output path), creates a Hadoop Job, registers all classes and output types, and submits the job to the cluster.

```
package com.hadoop.mapreduce;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class ContributionsJob {
    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: ContributionsJob <input path> <output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Employee Contributions");

        job.setJarByClass(ContributionsJob.class);

        job.setMapperClass(ContributionsMapper.class);
        job.setReducerClass(ContributionsReducer.class);

        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(job, new Path(args[0]));
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Item	Detail
Class	ContributionsJob
Entry point	main(String[] args)
Arguments	args[0] — HDFS input path; args[1] — HDFS output path
Job name	"Employee Contributions"
Mapper class	ContributionsMapper
Reducer class	ContributionsReducer
Output key	Text (employee ID)
Output value	IntWritable (total contributions)

4. How to Build and Run

4.1 Build

```
mvn clean package
```

This produces `target/EmployeeContributions-1.0-SNAPSHOT.jar`.

4.2 Run on Hadoop

```
hadoop jar target/EmployeeContributions-1.0-SNAPSHOT.jar \  
  com.hadoop.mapreduce.ContributionsJob \  
  /input/contributions.csv \  
  /output/contributions_result
```

4.3 Expected Input Format

The input CSV file must have (at minimum) four columns:

```
year,<col2>,employee_id,contribution_amount,...
```

Example:

```
year,department,emp_id,amount  
2023,Engineering,E001,5000  
2023,Marketing,E002,3000  
2024,Engineering,E001,7000
```

4.4 Expected Output Format

Each output line contains an employee ID and their **total** contribution amount:

```
E001    12000  
E002    3000
```